The `MethodInfo` property can be used to control how the filter is created based on the definition of the endpoint handler. Listing 5.11 shows how you can loop through the parameters to check for the details you need—a `string id` parameter, in this case—and customize the filter function you return.

If you find all these method signatures to be confusing, I don't blame you. Remembering the difference between an `EndpointFilterFactoryContext` and `EndpointFilterInvocationContext` and then trying to satisfy the compiler with your lambda methods can be annoying. Sometimes, you yearn for a good ol' interface to implement. Let's do that now.

5.4.4 Implementing the IEndpointFilter interface

Creating a lambda method for `AddEndpointFilter()` that satisfies the compiler can be a frustrating experience, depending on the level of support your integrated development environment (IDE) provides. In this section you'll learn how to sidestep the issue by defining a class that implements `IEndpointFilter` instead.

You can implement `IEndpointFilter` by defining a class with an `InvokeAsync()` that has the same signature as the lambda defined in listing 5.9. The advantage of using `IEndpointFilter` is that you get IntelliSense and autocompletion for the method signature. The following listing shows how to implement an `IEndpointFilter` class that's equivalent to listing 5.9.

Listing 5.12 Implementing `IEndpointFilter`